EXERCISE SESSION 4 – Wednesday 15 October 2025 (2 hours)
SQL - SOLUTIONS

STUDENT (StudentID, FirstName, L**AS**tName, Address, City)
TEACHER (TeacherID, FirstName, L**AS**tName, City, Ph**ON**e, Salary)
COURSE (Code, Name, Faculty, NumberOfCredits, ProfessorID, NumberOfStudents)
EXAM (CourseCode, StudentID, Grade)

Solve the following queries IN SQL:

20. Find the student IDs of students who earned more 30s with honors than 30s.

```
SELECT E.StudentID
FROM EXAM AS E
WHERE E.Grade=33
GROUP BY E.StudentID
HAVING COUNT(*) > ( SELECT COUNT(*)
                    FROM EXAM AS E2
                    WHERE E.StudentID=E2.StudentID AND E2.Grade=30)
```

21. Find the student ID, first name, and last name of students who passed the exams in all courses.

```
SELECT S.StudentID, FirstName, LASTName
FROM STUDENT AS S, EXAM AS E
WHERE S.StudentID=E.StudentID
GROUP BY S.StudentID, FirstName, LASTName
HAVING COUNT(*) = (  SELECT COUNT(*)
                     FROM COURSE)
```

Alternatively:

```
SELECT S.StudentID, S.FirstName, S.LASTName
FROM STUDENT AS S
WHERE NOT EXISTS (  SELECT *
                    FROM COURSE AS C
                    WHERE S.StudentID NOT IN ( SELECT E.StudentID
                                               FROM EXAM AS E
                                               WHERE E.CourseCode=C.Codice ))
```

22. Find the average grade AND student ID of students whose average grade is higher than the overall average grade.

```
SELECT StudentID, AVG(Grade)
FROM EXAM
GROUP BY StudentID
HAVING AVG(Grade) > (SELECT AVG(Grade) FROM EXAM)
```

STUDENT (SId, Name, Birthdate, City, Sex)
COURSE (CourseId, Title, Credits, Year, Professor) Year is 1 for exams of the 1st year
EXAM (SId, CId, Date, Grade) Grade is between 18 and 30

1. The (weighted) GPA for each year of each student who passed at least 5 exams (more precisely: "who passed at least 5 exams overall", not "5 per year").

```
SELECT SId, year, SUM(Grade*Credits)/SUM(Credits) AS weightedGpa
FROM EXAM JOIN COURSE ON CId = courseid NATURAL JOIN student
WHERE SId IN ( SELECT SId
               FROM EXAM
               GROUP BY SId
               HAVING COUNT(*) >= 5 )
GROUP BY SId, year
```

Alternatively:

```
SELECT SId, year, SUM(Grade*Credits)/SUM(Credits) AS weightedGpa
FROM EXAM JOIN COURSE ON CId = courseid NATURAL JOIN student S
WHERE 5 <= ( SELECT COUNT(*)
             FROM EXAM
             WHERE SId = S.SId)
GROUP BY SId, year
```

It is fundamental to realize that the following version is WRONG since it answers to the other interpretation (GPA for each year of each student who passed at least 5 exams in the year).

```
SELECT SId, year, SUM(Grade*Credits)/SUM(Credits) AS weightedGpa
FROM EXAM JOIN COURSE ON CId = courseid NATURAL JOIN STUDENT
GROUP BY SId, year
HAVING COUNT(*) >= 5;
```

2. Students who never got the same grade twice

```
SELECT SId, Name
FROM STUDENT
WHERE SId NOT IN ( SELECT SId
                   FROM EXAM NATURAL JOIN STUDENT
                   GROUP BY SId
                   HAVING COUNT(DISTINCT Grade) < COUNT(*) )
```

Alternatively, we can just change the predicate in the HAVING clause. Please note that these two versions are not equivalent, since in the first case we are considering also students that have not done any exam, while in the following we are considering only students with at least one exam.

```
SELECT SId, Name
FROM EXAM NATURAL JOIN STUDENT
GROUP BY SId
HAVING COUNT(DISTINCT Grade) = COUNT(*);
```

Or we can complement w.r.t. the opposite condition, identifying the students with "double" grades

```
SELECT SId, Name
FROM STUDENT
WHERE SId NOT IN ( SELECT SId
FROM EXAM
GROUP BY SId, Grade
HAVING COUNT(*) > 1 );
```

3. Students who always got the very same grade in all their exams

```
SELECT SId, Name
FROM EXAM NATURAL JOIN STUDENT
GROUP BY SId
HAVING COUNT(DISTINCT Grade) = 1;
```

or, more creatively:

```
SELECT SId, Name
FROM EXAM NATURAL JOIN STUDENT
GROUP BY SId
HAVING MIN(Grade) = MAX(Grade);
```

What if we also want to show the grade? The constraint on the unicivocity of the values prevents us from adding "grade" to the target list, even if the value for the selected groups is actually unique! The simplest solution to this syntactic restriction is to extract an aggregate value of the grade, which is always syntactically ok. max, min, and avg are equally apt.

```
SELECT SId, Name, MAX(Grade) AS TheOnlyGradeEver
FROM EXAM NATURAL JOIN STUDENT
GROUP BY SId
HAVING COUNT(DISTINCT Grade) = 1;
```

4. Students who got all possible different grades

```
SELECT SId, Name
FROM EXAM NATURAL JOIN STUDENT
GROUP BY SId
HAVING COUNT(DISTINCT Grade) = ( SELECT COUNT(DISTINCT Grade)
                                 FROM EXAM );
```

The solution above assumes that all grades have been given at least once in "history". A reasonable assumption that makes the formulation independent of any a priori assumption on the grading system (min, max, and variety of grades).

5. Id and name of the students who passed more exams in the second year than exams from the first year.

```
SELECT SId, Name
FROM EXAM e NATURAL JOIN STUDENT JOIN COURSE ON CId = courseid
WHERE year = 2
GROUP BY SId
HAVING COUNT(*) > ( SELECT COUNT(*)
                    FROM EXAM JOIN COURSE ON CId = courseid
                    WHERE year = 1 AND SId = e.SId );
```

Would it be the same if we asked for exams of year 1 in the outer query and exams of year 2 in the nested query (using the "<" comparison)? No, because I would restrict the result to those students who have at least one exam from year 1! Instead, students who passed 0 exams in year 1 (and have at least one exam in year 2) should be part of the result.

BD1 (exam 12-01-2022)

The following schema stores information about time-based competitions of the Winter Olympic Games.
ATHLETE (AthleteID, Name, Surname, Country, BirthDate)
COMPETITION (CompID, Discipline, Date, ItIsTheFinal)
PARTICIPATION(AthleteID, CompID, Time)

2. Find the ID, Name and Surname of the athletes who took part in at least two competitions, but never in a final.

```
select AthleteID, Name, Surname
from ATHLETE natural join PARTICIPATION
where AthleteID not in ( select AthleteID
                         from PARTICIPATION natural join COMPETITION
                         where ItIsTheFinal = 'yes' )
group by AthleteID
having count(*) > 1
```

3. For each Country, show the number of gold medals obtained by its athletes. Athletes obtain a gold medal when they obtain the best (shortest) time in a final.

```
select Country, count(*) as NumOfGolds
from ATHLETE natural join PARTICIPATION P natural join COMPETITION
where ItIsTheFinal = 'yes' and Time = ( select min(Time)
                                        from PARTICIPATION
                                        where CompID = P.CompID )
group by Country
```